Assignment 1
Arseniy Uspenskiy
USPARS001

Overview: The total codebase consists of 6 classes, three operate as main methods used for running specific parts of the code and provide some output to the user, and three create objects that store the *SA Place Data.*

PlaceNameEntry:

- Saves data of a single place, id, name, etc.
- Has multiple constructors allowing creation from an array or individual elements, as well as having a copy constructor
- Used by PlaceNameArray and PlaceNameBST as both classes store instances of PlaceNameEntry as their values

PlaceNameArray:

- Array that stores instances of the PlaceNameEntry class not allowing for duplicates
- Allows the user to continuously load PlaceNameEntries and to search for a specific entry using the name of the place
- Is run by the user in the PlaceSearchArray class and is also automatically used by the PlaceExperiment class

PlaceSearchArray:

- Allows the user a set of actions that can be done to an instance of PlaceNameArray. Load, Search, Print(Creative Extension) Help(Creative Extension), and Quit. (case insensitive)

PlaceNameBST:

- BST that stores instances of the PlaceNameEntry class not allowing for duplicates
- Allows the user to continuously load PlaceNameEntries and to search for a specific entry using the name of the place
- Is run by the user in the PlaceSearchBST class and is also automatically used by the PlaceExperiment class

PlaceSearchBST:

- Allows the user a set of actions that can be done to an instance of PlaceNameBST. Load, Search, Print(Creative Extension), Help(Creative Extension), and Quit. (case insensitive)

PlaceExperiment:

- The program runs an experiment comparing the number of comparisons made when searching for an entry in an array, BST, BST(sorted) and BST(optimal) with entries from 1000 to 10000 in increments of 1000.

Creative Extensions:

- In both PlaceSearchArray and PlaceSearchBST the user has an additional commands print which prints all the loaded data and help that prints out a simple explanation of what the class does

PlaceSearchArray:

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchArray
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
L
Please enter the path to the file: files/SAplaceNames.csv
Please enter how many places to load: 32
32 places loaded.
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Search
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
161007 Citrusdal Cederberg Western Cape 7177
Number of Comparisons: 20
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
s
Please enter the name of the place to find: Constantia
---Searching for: Constantia---
Place not found in database
Number of Comparisons: 32
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
help
- PlaceSearchArray:
Program allows the user to dynamically load Place data into an array
and then lets the user to search the array for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Q
```

PlaceSearchBST

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchBST
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
L
Please enter the path to the file: files/SAplaceNames.csv
Please enter how many places to load: 32
32 places loaded.
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
search
Please enter the name of the place to find: Constantia
---Searching for: Constantia---
Place not found in database
Number of Comparisons: 7
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Search
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
161007 Citrusdal Cederberg Western Cape 7177
Number of Comparisons: 7
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
h
- PlacesSearchBST
 Program allows the user to dynamically load Place data into a binary search tree
and then lets the user to search for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Quit
```

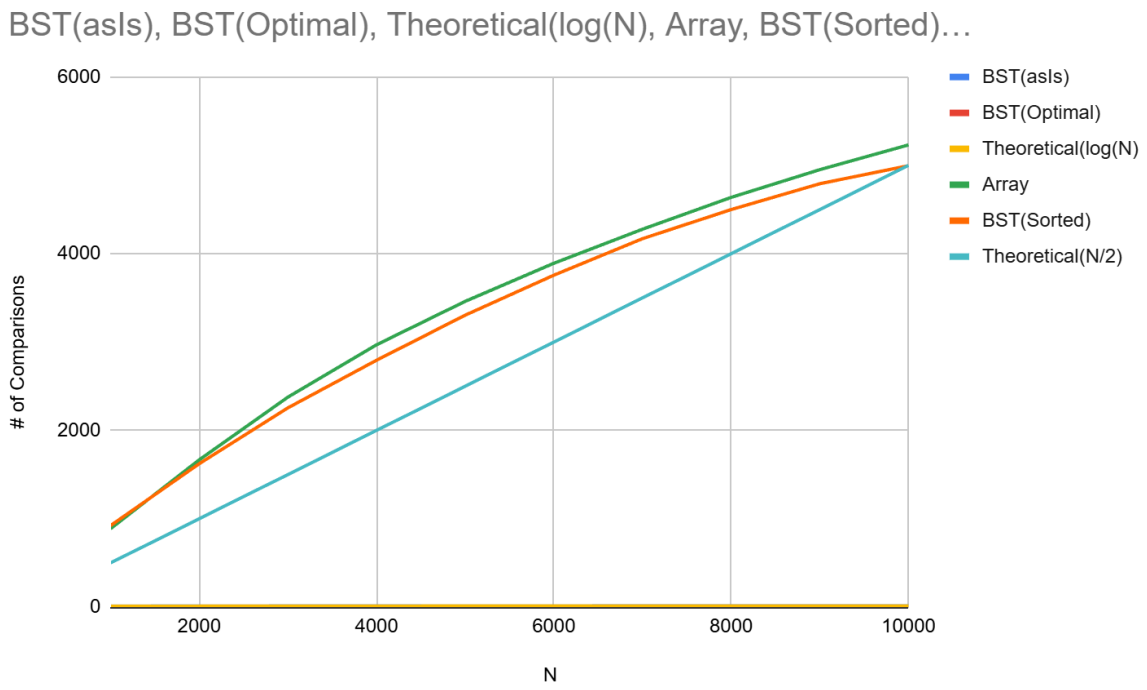
Edge Cases:

- Searching for last or first element
- Searching for not existent element
- Searching an empty instance
- Searching with very small N
- Searching with very large N

Data Table:

N	Array	BST(asIs)	BST(Sorted)	BST(Optimal)
1000	887.3	11.6	922.3	9.9
2000	1666.5	12.7	1622.5	10.9
3000	2376.3	13.4	2255	11.4
4000	2969	13.9	2794	11.8
5000	3460.9	14.2	3304.2	12.1
6000	3891.8	14.3	3757	12.2
7000	4277.4	14.6	4170.3	12.3
8000	4637.7	14.8	4499.7	12.5
9000	4951.6	14.9	4792.3	12.7
10000	5231.6	15	4996.7	12.7

Complete Graph:



Method: Using a set file of 50 search queries covering both existing and non-existing ones as well as edge cases of being at the beginning and end of a set of N data points. The search queries were run using an Array, BST(from base data), BST(from sorted data), and a BST(optimally ordered data). The test is run on each data structure 10 times with increasing the number of entries starting from 1000 and ending on 10000 entries, increasing by increments of 1000.

Results: Based on both the table and the graph each data structure experienced the following time complexity. Array : O(N), BST(asIs) : O(log(N)), BST(sorted) : O(N), BST(optimal) : O(log(N)). These values are consistent with the expected theoretical values that were covered in class.

Conclusion: The data shows a key element of how a Binary Search Tree functions as seen by the fact that two of the BST implementations experienced a complexity consistent with O(log(N)) meanwhile BST(sorted) had a complexity consistent with O(N). This is because the order in which data is inserted into a binary search tree matters and when sorted data is inserted into a BST, every new data point is always placed as the right node of the previous inserted data point creating what is essentially just a linked list. Similarly though less noticeable there is a consistent gap in between the BST(asIs) and BST(optimal). This is also caused due to the order the data is inserted as over time due to non-optimal inputs the as is tree becomes unbalanced and one side of the tree ends up having a larger height than the other resulting in slightly more search comparisons being required, especially at larger N.

AI disclosure: AI was only used towards the end of the assignment to identify potential flaws that may have gone unnoticed in production as well as help with structuring java docs. The tool used was Chat GPT thinking 5.4, no AI code or writing was inserted into the project directly.